UNIT -2

INTRODUCTION TO RELATIONAL MODEL

Relational Model in DBMS

E.F. Codd proposed the relational Model to model data in the form of relations or tables. After designing the conceptual model of the Database using ER diagram, we need to convert the conceptual model into a relational model which can be implemented using any RDBMS language like Oracle SQL, MySQL, etc. So we will see what the Relational Model is.

What is the Relational Model?

The relational model represents how data is stored in Relational Databases. A relational database consists of a collection of tables, each of which is assigned a unique name. Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE, and AGE shown in the table.

Table Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI		18

important Terminologies

- **Attribute:** Attributes are the properties that define an entity. e.g.; **ROLL_NO**, **NAME, ADDRESS**
- **Relation Schema:** A relation schema defines the structure of the relation and represents the name of the relation with its attributes. e.g.; STUDENT (ROLL_NO, NAME, ADDRESS, PHONE, and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation, it is called Relational Schema.
- **Tuple:** Each row in the relation is known as a tuple. The above relation contains 4 tuples, one of which is shown as:

1	RAM	DELHI	9455123451	18

- **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. Table 1 shows the relation instance of STUDENT at a particular time. It can change whenever there is an insertion, deletion, or update in the database.

- **Degree:** The number of attributes in the relation is known as the degree of the relation. The **STUDENT** relation defined above has degree 5.
- **Cardinality:** The number of tuples in a relation is known as cardinality. The **STUDENT** relation defined above has cardinality 4.
- **Column:** The column represents the set of values for a particular attribute. The column **ROLL_NO** is extracted from the relation STUDENT.

ROLL_NO
1
2
3
4

- **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by blank space. e.g.; PHONE of STUDENT having ROLL_NO 4 is NULL.
- **Relation Key:** These are basically the keys that are used to identify the rows uniquely or also help in identifying tables. These are of the following types.
    - Primary Key
    - Candidate Key
    - Super Key
    - Foreign Key
    - Alternate Key
    - Composite Key

Constraints in Relational Model

While designing the Relational Model, we define some conditions which must hold for data present in the database are called Constraints. These constraints are checked before performing any operation (insertion, deletion, and updation) in the database. If there is a violation of any of the constraints, the operation will fail.

Domain Constraints

These are attribute-level constraints. An attribute can only take values that lie inside the domain range. e.g.; If a constraint AGE>0 is applied to STUDENT relation, inserting a negative value of AGE will result in failure.

Key Integrity

Every relation in the database should have at least one set of attributes that defines a tuple uniquely. Those set of attributes is called keys. e.g.; ROLL_NO in STUDENT is key. No two students can have the same roll number. So a key has two properties:

- It should be unique for all tuples.
- It can't have NULL values.

Referential Integrity

When one attribute of a relation can only take values from another attribute of the same relation or any other relation, it is called referential integrity. Let us suppose we have 2 relations

Table Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE	BRANCH_CODE
1	RAM	DELHI	9455123451	18	CS
2	RAMESH	GURGAON	9652431543	18	CS
3	SUJIT	ROHTAK	9156253131	20	ECE
4	SURESH	DELHI		18	IT

Table Branch

BRANCH_CODE	BRANCH_NAME
CS	COMPUTER SCIENCE
IT	INFORMATION TECHNOLOGY
ECE	ELECTRONICS AND COMMUNICATION ENGINEERING
CV	CIVIL ENGINEERING

BRANCH_CODE of STUDENT can only take the values which are present in BRANCH_CODE of BRANCH which is called referential integrity constraint. The relation which is referencing another relation is called REFERENCING RELATION (STUDENT in this case) and the relation to which other relations refer is called REFERENCED RELATION (BRANCH in this case).

Keys:

A key refers to an attribute/a set of attributes that help us identify a row (or tuple) uniquely in a table (or relation). A key is also used when we want to establish relationships between the different columns and tables of a relational database. The individual values present in a key are commonly referred to as key values.

Why do we require Keys in DBMS?

We use a key for defining various types of integrity constraints in a database. A table, on the other hand, represents a collection of the records of various events for any relation. Now, there might be thousands of these records, and some of these might even be duplicated.

Thus, we need a way in which one can identify all of these records uniquely and separately, i.e., without any duplicates. This hassle is removed with the help of keys.

For example, let us consider a database of all the students who are studying in a college. What attribute of all the students, according to you, will identify each of these people uniquely? We can refer to these students by their names, departments, sections, and year. Similarly, we can also mention only the university roll number and fetch all the other details based on that roll number.

The keys in DBMS can be a combination of multiple attributes (or columns), or they can be just one single attribute. The primary motive of the keys is to provide every record with a unique identity of its own.

Different types of Keys in DBMS

Keys are of seven broad types in DBMS:

1. Super Key
2. Candidate key
3. Primary Key
4. Foreign Key
5. Alternate Key
6. Composite Key
7. Unique Key

Super Key and Candidate key:

A super key is a set of one or more attributes (columns), which can uniquely identify a row in a table.

How candidate key is different from super key?

Candidate keys are selected from the set of super keys, the only thing we take care while selecting candidate key is: It should not have any redundant attribute. That's the reason they are also termed as minimal super key.

Let's take an example to understand this:
Table: Employee

```
Emp_SSN          Emp_Number     Emp_Name
---------   ----------   --------
123456789        226        Steve
999999321        227        Ajeet
```

888997212	228	Chaitanya
777778888	229	Robert

Super keys: The above table has following super keys. All of the following sets of super key are able to uniquely identify a row of the employee table.

- {Emp_SSN}
- {Emp_Number}
- {Emp_SSN, Emp_Number}
- {Emp_SSN, Emp_Name}
- {Emp_SSN, Emp_Number, Emp_Name}
- {Emp_Number, Emp_Name}

Candidate Keys: As mentioned in the beginning, a candidate key is a minimal super key with no redundant attributes. The following two set of super keys are chosen from the above sets as there are no redundant attributes in these sets.

- {Emp_SSN}
- {Emp_Number}

Only these two sets are candidate keys as all other sets are having redundant attributes that are not necessary for unique identification.

Super key vs Candidate Key

I have been getting lot of comments regarding the confusion between super key and candidate key. Let me give you a clear explanation.
1. First you have to understand that all the candidate keys are super keys. This is because the candidate keys are chosen out of the super keys.
2. How we choose candidate keys from the set of super keys? We look for those keys from which we cannot remove any fields. In the above example, we have not chosen {Emp_SSN, Emp_Name} as candidate key because {Emp_SSN} alone can identify a unique row in the table and Emp_Name is redundant.

<u>Primary Key:</u>

There are certain keys in DBMS that are used for different purposes, from which the most commonly known is the Primary Key.

A Primary Key is the minimal set of attributes of a table that has the task to uniquely identify the rows, or we can say the tuples of the given particular table.

A primary key of a relation is one of the possible candidate keys which the database designer thinks it's primary. It may be selected for convenience, performance and many other reasons. The choice of the possible primary key from the candidate keys depend upon the following conditions.

- **Minimal:** The primary key should be composed of the minimum number of attributes that satisfy unique occurrences of the tuples. So if one candidate key is formed using two attributes and another using a single attribute then the one with the single attribute key should be chosen as the primary key.
- **Accessible:** The primary key used should be accessible by anyone who wants to use it. The user must easily insert, access or delete a tuple using it.
- **NON NULL Value:** The primary key must have a non-null value for each tuple of the relation, which is required for the identification of the tuple.
- **Time Variant:** The values of the primary key must not change or become null during the time of a relation.
- **Unique:** The value of the primary key must not be duplicated in any of the tuples of a relation.

Syntax for creating primary key constraint:

<column_name><datatype> Primary key;

Properties of a Primary Key:

- A relation can contain only one primary key.
- A primary key may be composed of a single attribute known as single primary key or more than one attribute known as composite key.
- A primary key is the minimum super key.
- The data values for the primary key attribute should not be null.
- Attributes which are part of a primary key are known as Prime attributes.
- Primary key is always chosen from the possible candidate keys.
- If the primary key is made of more than one attribute then those attributes are irreducible.
- We use the convention that the attribute that form primary key of relation is underlined.
- Primary key cannot contain duplicate values.

For example: When we store the registration details of the students in the database, we find the registration number field unique and assign the primary key to the field. Also, for an employee table, we set the primary key on the employee Id of the table.

et's understand it practically:

Below is the table named **STUDENT_DETAILS**, where Roll_no, Name, and Marks are the specified attributes of it.

STUDENT_DETAILS

Roll_no	Name	Marks
101	X	34
102	Y	46
103	Z	94

Primary Key

As we know that from these three attributes, the Roll_no attribute is the one that can uniquely identify other two attributes of the table as each student is provided with a unique roll number in every organization. So, we can set the primary key constraint on the Roll_no column.

What if we set Name as Primary Key?

If we set the primary key on the Name attribute, it will be invalid because two or more students can have the same name. So, if we set the primary key on Name and try to enter the same name for two students, it will display an error. That's why we cannot set the Name attribute as the primary key.

What if we set Marks as Primary Key?

If we set the primary key on the Marks attribute, then it will be an inappropriate approach because two or more students can score similar marks in a subject. Thus, if we set the Marks attribute as the primary key, we will not be able to enter the same score for other entity. That's why we cannot set the primary key for the Marks attribute.

Foreign Key

A foreign key is the one that is used to link two tables together via the primary key. It means the columns of one table points to the primary key attribute of the other table. It further means that if any attribute is set as a primary key attribute will work in another table as a foreign key attribute. But one should know that a foreign key has nothing to do with the primary key.

Use of Foreign Key

The use of a foreign key is simply to link the attributes of two tables together with the help of a primary key attribute. Thus, it is used for creating and maintaining the relationship between the two relations.

Example of Foreign Key

Let's discuss an example to understand the working of a foreign key.

Consider two tables **Student** and **Department** having their respective attributes as shown in the below table structure:

Student

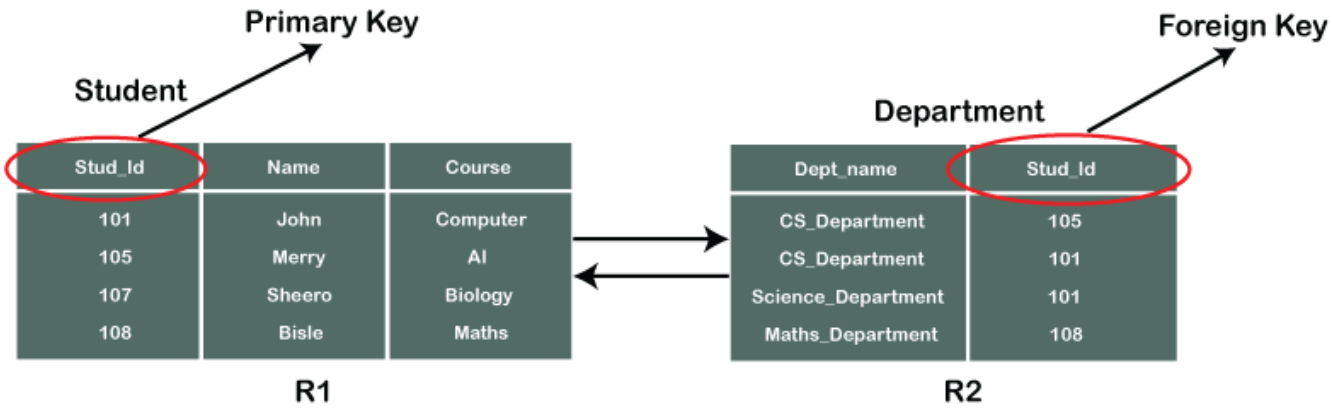
Stud_Id	Name	Course
101	John	Computer
105	Merry	AI
107	Sheero	Biology
108	Bisle	Maths

Department

Dept_name	Stud_Id
CS_Department	105
CS_Department	101
Science_Department	101
Math_Department	108

In the tables, one attribute, you can see, is common, that is **Stud_Id**, but it has different key constraints for both tables. In the Student table, the field Stud_Id is a **primary key** because it is uniquely identifying all other fields of the Student table. On the other hand, Stud_Id is a **foreign key** attribute for the Department table because it is acting as a primary key attribute for the Student table. It means that both the Student and Department table are linked with one another because of the Stud_Id attribute.

In the below-shown figure, you can view the following structure of the relationship between the two tables.



Note: Referential Integrity in DBMS is developed from the concept of the foreign key. It is clear that a primary key is an alone existing key and a foreign key always reference to a primary key in some other table, in which the table that contains the primary key is known as the referenced table or parent table for the other table that is having the foreign key.

Creating Foreign Key constraint

On CREATE TABLE

Below is the syntax that will make us learn the creation of a foreign key in a table:

```
CREATE TABLE Department (
    Dept_name varchar (10),
    Stud_Id int,
```

```
    FOREIGN KEY (Stud_Id) REFERENCES Student (Stud_Id)
);
```

Alternate Key

An alternate is a secondary candidate key that is capable of identifying a row uniquely. However, such a key is not used as a primary key because, as we have discussed in our previous section that out of all the generated candidate keys, only one key is selected as the primary key. Thus, the other remaining keys are known as **Alternate Keys** or **Secondary Keys**.

Composite Key

Composite key is a combination of two or more columns in a table that uniquely identify each row in the table. When these columns are combined, then the uniqueness of a row is guaranteed, but when the columns are taken individually, then it does not guarantee the uniqueness.

It can also be called a primary key made by the combination of two or more columns to uniquely identify every record in a table.

Note :
Primary key, super key, candidate key, or any other key can be called as the composite key if it consists of two or more columns to uniquely identify the records in a table.

Unique Key:

A unique key in DBMS is used to uniquely identify a tuple in a table and is used to prevent duplicity of the values in a table.

Role of Unique Key

A unique key is used to remove the duplicity of values in a table. However, the usage of a primary key is the same, but there is a difference between both keys. A primary key cannot take a **NULL** value, but a unique key can have one **NULL** value as its value.

How Unique Key is different from Primary Key

There are the following difference points that will let us understand the difference between a Unique key and a Primary key:

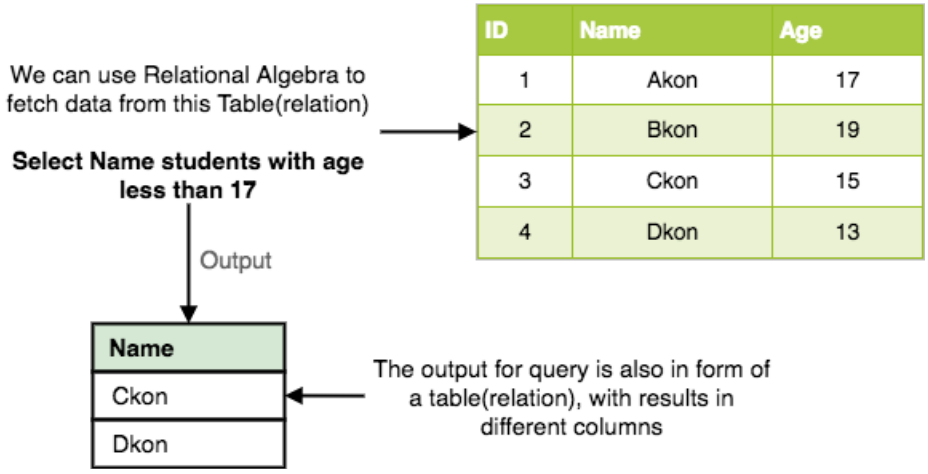
- A primary key field can never have a **NULL** value as its value, whereas a unique key can have one of its field values as **NULL**.
- A table cannot have more than one primary key, but a table can have more than one unique key.
- There are different types of relational databases, but in some of these databases, the primary key can generate a clustered index by default. On the other hand, a unique key is able to generate a non-clustered index by default.

These explained some of the differences that will help us to differentiate between both keys.

Note: The SQL standards believe that the unique key does not satisfy or guarantees the uniqueness of those rows having NULL as its value. However, other RDBMS does not follow the SQL standards.

Relational Algebra :

- Relational Algebra is procedural query language, which takes Relation as input and generates relation as output. Relational algebra mainly provides theoretical foundation for relational databases and SQL.
- Relational algebra is a procedural query language, it means that it tells what data to be retrieved and how to be retrieved.
- Relational Algebra works on the whole table at once, so we do not have to use loops etc to iterate over all the rows (tuples) of data one by one.
- All we have to do is specify the table name from which we need the data, and in a single line of command, relational algebra will traverse the entire given table to fetch data for you.



Basic/Fundamental Operations:

1.Select (σ)
2. Project (Π)
3. Union (U)
4. Set Difference (-)
5. Cartesian product (X)
6. Rename (ρ)

1. Select Operation (σ) :This is used to fetch rows (tuples) from table(relation) which satisfies a given condition.

Syntax: σp(r)

- σ is the predicate
- r stands for relation which is the name of the table
- p is prepositional logic

ex: σage > 17 (Student)

This will fetch the tuples(rows) from table Student, for which age will be greater than 17.

σage > 17 and gender = 'Male' (Student)
This will return tuples(rows) from table Student with information of male students, of age more than 17.

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input:
σ BRANCH_NAME="perryride" (LOAN)

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

2.Project Operation (Π):

- Project operation is used to project only a certain set of attributes of a relation. In simple words, If you want to see only the names all of the students in the Student table, then you can use Project Operation.
- It will only project or show the columns or attributes asked for, and will also remove duplicate data from the columns.

Syntax of Project Operator (Π)
Π column_name1, column_name2, ..., column_nameN(table_name)
Example:
ΠName, Age(Student)
Above statement will show us only the Name and Age columns for all the rows of data in Student table.

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison
Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye
Jhonson	Alma	Brooklyn
Brooks	Senator	Brooklyn

Input:

Π NAME, CITY (CUSTOMER)

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye
Johnson	Brooklyn
Brooks	Brooklyn

3.Union Operation (∪):

- This operation is used to fetch data from two relations(tables) or temporary relation(result of another operation).
- For this operation to work, the relations(tables) specified should have same number of attributes(columns) and same attribute domain. Also the duplicate tuples are autamatically eliminated from the result.

Syntax: A ∪ B

ΠStudent(RegularClass) ∪ ΠStudent(ExtraClass)

Example:

DEPOSITOR RELATION

CUSTOMER_NAME	ACCOUNT_NO
Johnson	A-101
Smith	A-121
Mayes	A-321
Turner	A-176
Johnson	A-273
Jones	A-472
Lindsay	A-284

BORROW RELATION

CUSTOMER_NAME	LOAN_NO
Jones	L-17
Smith	L-23
Hayes	L-15
Jackson	L-14
Curry	L-93
Smith	L-11

Input:
Π CUSTOMER_NAME (BORROW) ∪ Π CUSTOMER_NAME (DEPOSITOR)

Output:

CUSTOMER_NAME
Johnson
Smith
Hayes
Turner
Jones
Lindsay
Jackson
Curry
Williams
Mayes

4.Set Difference (-):
This operation is used to find data present in one relation and not present in the second relation. This operation is also applicable on two relations, just like Union operation.
Syntax: A - B
where A and B are relations.
For example, if we want to find name of students who attend the regular class but not the extra class, then, we can use the below operation:
ΠStudent(RegularClass) - ΠStudent(ExtraClass)

Input: Π CUSTOMER_NAME (BORROW) ∩ Π CUSTOMER_NAME (DEPOSITOR)

CUSTOMER_NAME
Smith
Jones

5.Cartesian Product (X):
This is used to combine data from two different relations(tables) into one and fetch data from the combined relation.
Syntax: A X B
For example, if we want to find the information for Regular Class and Extra Class which are conducted during morning, then, we can use the following operation:
σtime = 'morning' (RegularClass X ExtraClass)
For the above query to work, both RegularClass and ExtraClass should have the attribute time.
Notation: E X D

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT

1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Input:

EMPLOYEE X DEPARTMENT

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

5.Rename Operation (ρ):

This operation is used to rename the output relation for any query operation which returns result like Select, Project etc. Or to simply rename a relation(table)

Syntax: ρ(RelationNew, RelationOld)

The rename operation is used to rename the output relation. It is denoted by rho (ρ).

Example: We can use the rename operator to rename STUDENT relation to STUDENT1.

ρ(STUDENT1, STUDENT)

Join in DBMS:

- A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
- Join in DBMS is a binary operation which allows you to combine join product and selection in one single statement.
- The goal of creating a join condition is that it helps you to combine the data from two or more DBMS tables.

There are mainly two types of joins in DBMS:

1. Inner Joins: Theta, Natural, EQUI
2. Outer Join: Left, Right, Full

Theta (θ) Join

Theta join combines tuples from different relations provided they satisfy the theta condition. The join condition is denoted by the symbol **θ**.

Notation

$R1 \bowtie_{\theta} R2$

R1 and R2 are relations having attributes (A1, A2, .., An) and (B1, B2,.. ,Bn) such that the attributes don't have anything in common, that is $R1 \cap R2 = \Phi$.

Theta join can use all kinds of comparison operators.

Student

SID	Name	Std
101	Alex	10
102	Maria	11

Subjects

Class	Subject
10	Math
10	English
11	Music
11	Sports

Student_Detail –

$STUDENT \bowtie_{Student.Std = Subject.Class} SUBJECT$

Student_detail

SID	Name	Std	Class	Subject
101	Alex	10	10	Math

101	Alex	10	10	English
102	Maria	11	11	Music
102	Maria	11	11	Sports

Equijoin

When Theta join uses only **equality** comparison operator, it is said to be equijoin. The above example corresponds to equijoin.

Natural Join (⋈)

Natural join does not use any comparison operator. It does not concatenate the way a Cartesian product does. We can perform a Natural Join only if there is at least one common attribute that exists between two relations. In addition, the attributes must have the same name and domain.

Natural join acts on those matching attributes where the values of attributes in both the relations are same.

Courses		
CID	**Course**	**Dept**
CS01	Database	CS
ME01	Mechanics	ME
EE01	Electronics	EE

HoD	
Dept	**Head**
CS	Alex
ME	Maya
EE	Mira

Courses ⋈ HoD			
Dept	**CID**	**Course**	**Head**
CS	CS01	Database	Alex
ME	ME01	Mechanics	Maya
EE	EE01	Electronics	Mira

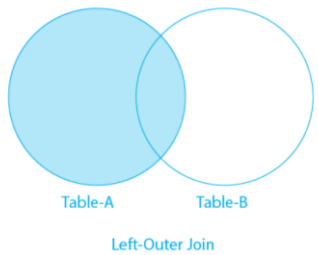
Outer Joins

Theta Join, Equijoin, and Natural Join are called inner joins. An inner join includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use outer

joins to include all the tuples from the participating relations in the resulting relation. There are three kinds of outer joins – left outer join, right outer join, and full outer join.

Left Outer Join(R ⟕ S)

All the tuples from the Left relation, R, are included in the resulting relation. If there are tuples in R without any matching tuple in the Right relation S, then the S-attributes of the resulting relation are made NULL.



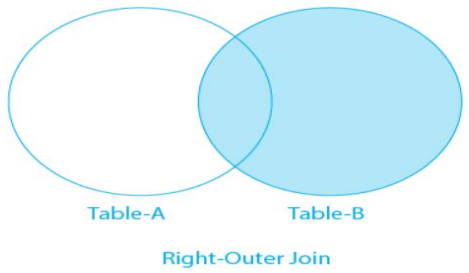
Left	
A	**B**
100	Database
101	Mechanics
102	Electronics

Right	
A	**B**
100	Alex
102	Maya
104	Mira

Courses ⟕ HoD			
A	**B**	**C**	**D**
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya

Right Outer Join: (R ⟖ S)

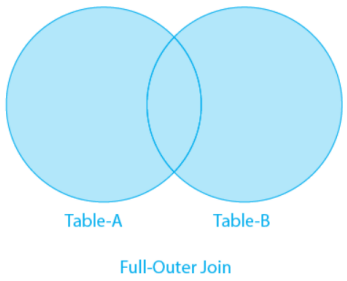
All the tuples from the Right relation, S, are included in the resulting relation. If there are tuples in S without any matching tuple in R, then the R-attributes of resulting relation are made NULL



Courses ⋈ HoD			
A	**B**	**C**	**D**
100	Database	100	Alex
102	Electronics	102	Maya
---	---	104	Mira

Full Outer Join: (R ⟗ S)

All the tuples from both participating relations are included in the resulting relation. If there are no matching tuples for both relations, their respective unmatched attributes are made NULL.



Courses ⋈ HoD			
A	**B**	**C**	**D**
100	Database	100	Alex
101	Mechanics	---	---
102	Electronics	102	Maya
---	---	104	Mira

Relational Calculus:

Relational calculus is a non-procedural query language that tells the system what data to be retrieved but doesn't tell how to retrieve it. Relational Calculus exists in two forms:

1. Tuple Relational Calculus (TRC)

2. Domain Relational Calculus (DRC)

Tuple Relational Calculus (TRC)

Tuple Relational Calculus (TRC) is a non-procedural query language used in relational database management systems (RDBMS) to retrieve data from tables. TRC is based on the concept of tuples, which are ordered sets of attribute values that represent a single row or record in a database table.
TRC is a declarative language, meaning that it specifies what data is required from the database, rather than how to retrieve it. TRC queries are expressed as logical formulas that describe the desired tuples.
Syntax: The basic syntax of TRC is as follows:
{ t | P(t) }

where t is a **tuple variable** and P(t) is a **logical formula** that describes the conditions that the tuples in the result must satisfy. The **curly braces {}** are used to indicate that the expression is a set of tuples.

P(t) may have various conditions logically combined with OR (∨), AND (∧), NOT(¬).
It also uses quantifiers:
∃ t ∈ r (Q(t)) = ”there exists” a tuple in t in relation r such that predicate Q(t) is true.
∀ t ∈ r (Q(t)) = Q(t) is true “for all” tuples in relation r.

For example, let’s say we have a table called “Employees” with the following attributes:

Employee ID
Name
Salary
Department ID

To retrieve the names of all employees who earn more than $50,000 per year, we can use the following TRC query:

{ t | Employees(t) ∧ t.Salary > 50000 }

In this query, the “Employees(t)” expression specifies that the tuple variable t represents a row in the “Employees” table. The “∧” symbol is the logical AND operator, which is used to combine the condition “t.Salary > 50000” with the table selection.

The result of this query will be a set of tuples, where each tuple contains the Name attribute of an employee who earns more than $50,000 per year.

TRC can also be used to perform more complex queries, such as joins and nested queries, by using additional logical operators and expressions.

While TRC is a powerful query language, it can be more difficult to write and understand than other SQL-based query languages, such as Structured Query Language (SQL). However, it is useful in certain applications, such as in the formal verification of database schemas and in academic research.
Tuple Relational Calculus is a **non-procedural query language,** unlike relational algebra. Tuple Calculus provides only the description of the query but it does not provide the methods to solve it. Thus, it explains what to do but not how to do it.

Domain Relational Calculus (DRC)

Domain Relational Calculus (DRC)

The second form of relation is known as Domain relational calculus. In domain relational calculus, filtering variable uses the domain of attributes. Domain relational calculus uses the same operators as tuple calculus. It uses logical connectives ∧ (and), ∨ (or) and ┐ (not). It uses Existential (∃) and Universal Quantifiers (∀) to bind the variable. The QBE or Query by example is a query language related to domain relational calculus.

Syntax:

1. { a1, a2, a3, ..., an | P (a1, a2, a3, ... ,an)}

Where

a1, a2 are attributes
P stands for formula built by inner attributes

For example:

1. {< article, page, subject > | ∈ javatpoint ∧ subject = 'database'}

Output: This query will yield the article, page, and subject from the relational javatpoint, where the subject is a database.

SQL Basic Structure

THE FORM OF A BASIC SQL QUERY

This section presents the syntax of a simple SQL query and explains its meaning through a *conceptual evaluation strategy*. A conceptual evaluation strategy is a way to evaluate the query that is intended to be easy to understand, rather than efficient. A DBMS would typically execute a query in a different and more efficient way.

Sid	*sname*	*rating*	*age*
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 5.1 An Instance *S*3 of Sailors

sid	*bid*	*day*
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Figure 5.2 An Instance *R*2 of Reserves

bid	*bname*	*color*
101	*Interlake*	*blue*
102	*Interlake*	*red*
103	*Clipper*	*green*
104	*Marine*	*red*

The basic form of an SQL query is as follows:

SELECT [DISTINCT] select-list FROM from-list WHERE qualification Such a query intuitively corresponds to a relational algebra expression involving selections, projections, and cross-products. Every query must have a SELECT clause, which specifies columns to be retained in the result, and a FROM clause, which specifies a cross-product of tables. The optional WHERE clause specifies selection conditions on the tables mentioned in the FROM clause. Let us consider a simple query.

(Q1) Find the names and ages of all sailors.

```
SELECT DISTINCT S.sname, S.age FROM Sailors S
```

The answer is a *set* of rows, each of which is a pair ⟨*sname, age*⟩. If two or more sailors have the same name and age, the answer still contains just one pair with that name and age. This query is equivalent to applying the projection operator of relational algebra. The answer to this query with and without the keyword DISTINCT on instance *S*3 of Sailors is shown in Figures. The only difference is that the tuple for Horatio appears twice if DISTINCT is omitted; this is because there are two sailors called Horatio and age 35.

sname	*age*
Dustin	45.0
Brutus	33.0
Lubber	55.5
Andy	25.5
Rusty	35.0
Horatio	35.0
Zorba	16.0
Art	25.5
Bob	63.5

Figure: Answer to above question

Sname	*age*
Dustin	45.0
Brutus	33.0
Lubber	55.5
Andy	25.5
Rusty	35.0
Horatio	35.0
Zorba	16.0
Horatio	35.0
Art	25.5
Bob	63.5

Figure :Answer to above without DISTINCT

(Q2) Find all sailors with a rating above 7.

SELECT S.sid, S.sname, S.rating, S.age FROM Sailors AS S WHERE S.rating > 7

We now consider the syntax of a basic SQL query in more detail.

- The from-list in the FROM clause is a list of table names. A table name can be followed by a range variable; a range variable is particularly useful when the same table name appears more than once in the from-list.
- The select-list is a list of (expressions involving) column names of tables named in the from-list. Column names can be prefixed by a range variable.
- The qualification in the WHERE clause is a boolean combination (i.e., an expression using the logical connectives AND, OR, and NOT) of conditions of the form *expression* op *expression*, where op is one of the comparison operators *{<, <=, = , <>, >=, >}*.2 An *expression* is a *column* name, a *constant*, or an (arithmetic or string) expression.
- The DISTINCT keyword is optional. It indicates that the table computed as an answer to this query should not contain *duplicates*, that is, two copies of the same row. The default is that duplicates are not eliminated.

The syntax of a basic SQL query, they don't tell us the *meaning* of a query. The answer to a query is itself a relation which is a *multiset* of rows in SQL whose contents can be understood by considering the following conceptual evaluation strategy:

1. Compute the cross-product of the tables in the from-list.
2. Delete those rows in the cross-product that fail the qualification conditions.
3. Delete all columns that do not appear in the select-list.
4. If DISTINCT is specified, eliminate duplicate rows.

(Q3) Find the names of sailors who have reserved boat number 103.

It can be expressed in SQL as follows.

SELECT S.sname FROM Sailors S, Reserves R WHERE S.sid = R.sid AND R.bid=103

Let us compute the answer to this query on the instances *R*3 of Reserves and *S*4 of Sailors shown in Figures 2.1 and 2.2, since the computation on our usual example instances (*R*2 and *S*3) would be unnecessarily tedious.

sid	sna	rati	age
22	Dus	7	45.0
31	Lub	8	55.5
58	Rust	10	35.0

sid	bid	day
22	101	10/10/96
58	103	11/12/96

Figure 2.1 Instance *R*3 of Reserves Figure 2.2 Instance *S*4 of Sailors

The first step is to construct the cross-product *S*4 × *R*3, which is shown in Figure 2.3.

sid	sname	rating	age	sid	bid	day
22	Dusti	7	45.0	22	101	10/10/96
22	Dusti	7	45.0	58	103	11/12/96
31	Lubbe	8	55.5	22	101	10/10/96
31	Lubbe	8	55.5	58	103	11/12/96
58	Rusty	10	35.0	22	101	10/10/96
58	Rusty	10	35.0	58	103	11/12/96

Figure 2.3 *S4 × R3*

The second step is to apply the qualification *S.sid = R.sid* AND *R.bid=103*. (Note that the first part of this qualification requires a join operation.) This step eliminates all but the last row from the instance shown in Figure 5.8. The third step is to eliminate unwanted columns; only *sname* appears in the SELECT clause. This step leaves us with the result shown in Figure 5.9, which is a table with a single column and, as it happens, just one row.

Sname

sname
rusty

Figure 2.4 Answer to Query Q1 on R3 and S4

Examples of Basic SQL Queries

Query Q1, which we discussed in the previous section, can also be expressed as follows:

- SELECT sname FROM Sailors S, Reserves R WHERE S.sid = R.sid AND bid=103Only the occurrences of *sid* have to be qualified, since this column appears in both the Sailors and Reserves tables. An equivalent way to write this query is:

SELECT sname FROM Sailors, Reserves WHERE Sailors.sid = Reserves.sid AND bid=103

(Q4) Find the sids of sailors who have reserved a red boat.

SELECT R.sid FROM Boats B, Reserves R WHERE B.bid = R.bid AND B.color = 'red'

This query contains a join of two tables, followed by a selection on the color of boats.We can think of B and R as rows in the corresponding tables that 'prove' that a sailor with sid R.sid reserved a red boat B.bid. On our example instances *R*2 and *S*3.

(Q5) Find the names of sailors who have reserved a red boat.

SELECT S.sname FROM Sailors S, Reserves R, Boats B WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'

This query contains a join of three tables followed by a selection on the color of boats. The join with Sailors allows us to find the name of the sailor who, according to Reserves tuple R, has reserved a red boat described by tuple B.

(Q6) Find the colors of boats reserved by Lubber.

SELECT B.color FROM Sailors S, Reserves R, Boats B WHERE S.sid = R.sid AND R.bid = B.bid AND S.sname = 'Lubber'

This query is very similar to the previous one. Notice that in general there may be more than one sailor called Lubber (since *sname* is not a key for Sailors); this query is still correct in that it will return the colors of boats reserved by *some* Lubber, if there are several sailors called Lubber

(Q7) Find the names of sailors who have reserved at least one boat.

SELECT S.sname FROM Sailors S, Reserves R WHERE S.sid = R.sid

The join of Sailors and Reserves ensures that for each selected *sname*, the sailor has made some reservation. (If a sailor has not made a reservation, the second step in the conceptual evaluation strategy would eliminate all rows in the cross-product that involve this sailor.)

Expressions and Strings in the SELECT Command

SQL supports a more general version of the select-list than just a list of columns. Each item in a select-list can be of the form *expression* AS *column name*, where *expression* is any arithmetic or string expression over column names (possibly prefixed by range variables) and constants.

(Q8) Compute increments for the ratings of persons who have sailed two different boats on the same day.

SELECT S.sname, S.rating+1 AS rating FROM Sailors S, Reserves R1, Reserves R2 WHERE S.sid = R1.sid AND S.sid = R2.sid AND R1.day = R2.day AND R1.bid <> R2.bid

Also, each item in a *qualification* can be as general as *expression1 = expression2*.

SELECT S1.sname AS name1, S2.sname AS name2 FROM Sailors S1, Sailors S2 WHERE 2*S1.rating = S2.rating-1.

(Q9) Find the ages of sailors whose name begins and ends with B and has at least three characters.

SELECT S.age FROM Sailors S WHERE S.sname LIKE 'B %B'

The only such sailor is Bob, and his age is 63.5.

Constraints in SQL:

Constraints are the rules that we can apply on the type of data in a table. That is, we can specify the limit on the type of data that can be stored in a particular column in a table using constraints.

The available constraints in SQL are:

- **NOT NULL**: This constraint tells that we cannot store a null value in a column. That is, if a column is specified as NOT NULL then we will not be able to store null in this particular column any more.
- **UNIQUE**: This constraint when specified with a column, tells that all the values in the column must be unique. That is, the values in any row of a column must not be repeated.
- **PRIMARY KEY**: A primary key is a field which can uniquely identify each row in a table. And this constraint is used to specify a field in a table as primary key.
- **FOREIGN KEY**: A Foreign key is a field which can uniquely identify each row in a another table. And this constraint is used to specify a field as Foreign key.
- **CHECK**: This constraint helps to validate the values of a column to meet a particular condition. That is, it helps to ensure that the value stored in a column meets a specific condition.
- **DEFAULT**: This constraint specifies a default value for the column when no value is specified by the user.

How to specify constraints?
We can specify constraints at the time of creating the table using CREATE TABLE statement. We can also specify the constraints after creating a table using ALTER TABLE statement.
Syntax:
Below is the syntax to create constraints using CREATE TABLE statement at the time of creating the table.

```
CREATE TABLE sample_table

(

column1 data_type(size) constraint_name,

column2 data_type(size) constraint_name,

column3 data_type(size) constraint_name,

....

);
```

sample_table: Name of the table to be created.
data_type: Type of data that can be stored in the field.
constraint_name: Name of the constraint. for example- NOT NULL, UNIQUE, PRIMARY KEY etc.

UNION, INTERSECT, AND EXCEPT

SQL provides three set-manipulation constructs that extend the basic query form presented earlier. Since the answer to a query is a multiset of rows, it is natural to consider the use of operations such as union, intersection, and difference. SQL supports these operations under the names UNION, INTERSECT, and EXCEPT. SQL also provides other set operations: IN (to check if an element is in a given set),op ANY,op ALL(to compare a value with the elements in a given set, using comparison operator op),

and EXISTS (to check if a set is empty). IN and EXISTS can be prefixed by NOT, with the obvious modification to their meaning. We cover UNION, INTERSECT, and EXCEPT in this section.

Consider the following query:

(Q1) Find the names of sailors who have reserved a red or a green boat.

SELECT S.sname FROM Sailors S, Reserves R, Boats B WHERE S.sid = R.sid AND R.bid = B.bid AND (B.color = 'red' OR B.color = 'green')

This query is easily expressed using the OR connective in the WHERE clause. However, the following query, which is identical except for the use of 'and' rather than 'or' in the English version, turns out to be much more difficult

(Q2) Find the names of sailors who have reserved both a red and a green boat.

```
SELECT S.sname FROM Sailors S, Reserves R1, Boats B1, Reserves R2, Boats B2 WHERE S.sid = R1.sid
AND R1.bid = B1.bid AND S.sid = R2.sid AND
R2.bid = B2.bid AND B1.color='red' AND B2.color = 'green'
```

We can think of R1 and B1 as rows that prove that sailor S.sid has reserved a red boat. R2 and B2 similarly prove that the same sailor has reserved a green boat. S.sname is not included in the result unless five such rows S, R1, B1, R2, and B2 are found.The OR query(Q1) can be rewritten as follows:

```
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
UNION
SELECT S2.sname
FROM Sailors S2, Boats B2, Reserves R2
WHERE S2.sid = R2.sid AND R2.bid = B2.bid AND B2.color = 'green'
```

This query says that we want the union of the set of sailors who have reserved red boats and the set of sailors who have reserved green boats. In complete symmetry, the AND query (Query Q2) can be rewritten as follows:

```
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
INTERSECT
SELECT S2.sname
FROM Sailors S2, Boats B2, Reserves R2
WHERE S2.sid = R2.sid AND R2.bid = B2.bid AND B2.color = 'green'
```

(Q3) Find the sids of all sailors who have reserved red boats but not green boats.

```
SELECT S.sid FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
EXCEPT SELECT S2.sid FROM Sailors S2, Reserves R2, Boats B2
WHERE S2.sid = R2.sid AND R2.bid = B2.bid AND B2.color = 'green'
```

Sailors 22, 64, and 31 have reserved red boats. Sailors 22, 74, and 31 have reserved green boats. Thus, the answer contains just the *sid* 64. Indeed, since the Reserves relation contains sid information, there is no need to look at the Sailors relation, and we can use the following simpler query:

```
SELECT R.sid FROM Boats B, Reserves R
WHERE R.bid = B.bid AND B.color = 'red'
EXCEPT SELECT R2.sid FROM Boats B2,
Reserves R2 WHERE R2.bid = B2.bid AND
B2.color = 'green'
```

Note that UNION, INTERSECT, and EXCEPT can be used on *any* two tables that are union-compatible, that is, have the same number of columns and the columns, taken in order, have the same types. For example, we can write the following query:

(Q4) Find all sids of sailors who have a rating of 10 or have reserved boat 104.

```
SELECT S.sid FROM Sailors S WHERE S.rating = 10
UNION
SELECT R.sid FROM Reserves R WHERE R.bid = 104
```

The first part of the union returns the *sid*s 58 and 71. The second part returns 22and 31. The answer is, therefore, the set of *sid*s 22, 31, 58, and 71. A final point to note about UNION, INTERSECT, and EXCEPT follows. In contrast to the default that duplicates are not eliminated unless DISTINCT is specified in the basic query form, the default for UNION queries is that duplicates *are* eliminated! To retain duplicates, UNION ALL must be used; if so, the number of copies of a row in the result is $m + n$, where m and n are the numbers of times that the row appears in the two parts of the union. Similarly, one version of INTERSECT retains duplicates the number of copies of a row in the result is $min(m, n)$—and one version of EXCEPT also retains duplicates—the number of copies of a row in the result is $m - n$, where m corresponds to the first relation.

NESTED QUERIES

A nested query is a query that has another query embedded within it; the embedded query is called a subquery.

Introduction to Nested Queries

As an example, let us rewrite the following query, which we discussed earlier, using a nested subquery:

(Q1) Find the names of sailors who have reserved boat 103.

```
SELECT S.sname
FROM Sailors S
WHERE S.sid IN ( SELECT R.sid
FROM Reserves R
```

```
WHERE R.bid = 103 )
```

The nested subquery computes the (multi)set of *sid*s for sailors who have reserved boat 103 (the set contains 22, 31, and 74 on instances *R*2 and *S*3), and the top-level query retrieves the names of sailors whose *sid* is in this set. The IN operator allows us to test whether a value is in a given set of elements; an SQL query is used to generate the set to be tested. Notice that it is very easy to modify this query to find all sailors who have *not* reserved boat 103—we can just replace IN by NOT IN!

(Q2) Find the names of sailors who have reserved a red boat.

```
SELECT S.sname
FROM Sailors S
WHERE S.sid IN ( SELECT R.sid
FROM Reserves R
WHERE R.bid IN ( SELECT B.bid
FROM Boats B
WHERE B.color = 'red' )
```

The innermost subquery finds the set of *bid*s of red boats (102 and 104 on instance *B*1). The subquery one level above finds the set of *sid*s of sailors who have reserved one of these boats. On instances *B*1, *R*2, and *S*3, this set of *sid*s contains 22, 31, and64. The top-level query finds the names of sailors whose *sid* is in this set of *sid*s. For the example instances, we get Dustin, Lubber, and Horatio.

(Q3) Find the names of sailors who have not reserved a red boat.

```
SELECT S.sname
FROM Sailors S
WHERE S.sid NOT IN ( SELECT R.sid
FROM Reserves R
WHERE R.bid IN ( SELECT B.bid
FROM Boats B
WHERE B.color = 'red' )
```

This query computes the names of sailors whose *sid* is *not* in the set 22, 31, and 64.

Correlated Nested Queries

In the nested queries that we have seen thus far, the inner subquery has been completely independent of the outer query. In general the inner subquery could depend on the row that is currently being examined in the outer query (in terms of our conceptual evaluation strategy). Let us rewrite the following query once more:

(Q1) Find the names of sailors who have reserved boat number 103.

```
SELECT S.sname
FROM Sailors S
WHERE EXISTS ( SELECT *
FROM Reserves R
WHERE R.bid = 103
```

AND R.sid = S.sid)

The EXISTS operator is another set comparison operator, such as IN. It allows us to test whether a set is nonempty. Thus, for each Sailor row *S*, we test whether the set of Reserves rows *R* such that *R.bid = 103* AND *S.sid = R.sid* is nonempty. If so, sailor *S* has reserved boat 103, and we retrieve the name. The subquery clearly depends on the current row *S* and must be re-evaluated for each row in Sailors. The occurrence of *S* in the subquery (in the form of the literal *S.sid*) is called a *correlation*, and such queries are called *correlated queries*.

Set-Comparison Operators

We have already seen the set-comparison operators EXISTS, IN, and UNIQUE, along with their negated versions. SQL also supports op ANY and op ALL, where op is one of the arithmetic comparison operators *{<, <=, =, <>, >=, >}*. (SOME is also available, but it is just a synonym for ANY.)

(Q1) Find sailors whose rating is better than some sailor called Horatio.

```
SELECT S.sid
FROM Sailors S
WHERE S.rating > ANY ( SELECT S2.rating
FROM Sailors S2
WHERE S2.sname = 'Horatio' )
```

If there are several sailors called Horatio, this query finds all sailors whose rating is better than that of *some* sailor called Horatio. On instance *S*3, this computes the *sid*s 31, 32, 58, 71, and 74. What if there were *no* sailor called Horatio? In this case the comparison *S.rating* > ANY . . . is defined to return false, and the above query returns an empty answer set. To understand comparisons involving ANY, it is useful to think of the comparison being carried out repeatedly. In the example above, *S.rating* is successively compared with each rating value that is an answer to the nested query. Intuitively, the subquery must return a row that makes the comparison true, in order for *S.rating* > ANY . . . to return true.

(Q2) Find sailors whose rating is better than every sailor called Horatio.

We can obtain all such queries with a simple modification to Query Q1: just replace ANY with ALL in the WHERE clause of the outer query. On instance *S*3, we would get the *sid*s 58 and 71. If there were no sailor called Horatio, the comparison *S.rating* > ALL . . . is defined to return true! The query would then return the names of all sailors. Again, it is useful to think of the comparison being carried out repeatedly. Intuitively, the comparison must be true for every returned row in order for *S.rating* > ALL . . . to return true. As another illustration of ALL, consider the following query:

(Q3) Find the sailors with the highest rating.

```
SELECT S.sid
FROM Sailors S
WHERE S.rating >= ALL ( SELECT S2.rating
FROM Sailors S2 )
```

The subquery computes the set of all rating values in Sailors. The outer WHERE condition is satisfied only when *S.rating* is greater than or equal to each of these rating values, i.e., when it is the largest rating

value. In the instance *S*3, the condition is only satisfied for *rating* 10, and the answer includes the *sid*s of sailors with this rating, i.e., 58 and 71. Note that IN and NOT IN are equivalent to = ANY and <> ALL, respectively.

More Examples of Nested Queries

Let us revisit a query that we considered earlier using the INTERSECT operator.

(Q1) Find the names of sailors who have reserved both a red and a green boat.

```
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
AND S.sid IN ( SELECT S2.sid
FROM Sailors S2, Boats B2, Reserves R2
WHERE S2.sid = R2.sid AND R2.bid = B2.bid
AND B2.color = 'green' )
```

As it turns out, writing this query (Q6) using INTERSECT is more complicated because we have to use *sid*s to identify sailors (while intersecting) and have to return sailor names:

```
SELECT S3.sname
FROM Sailors S3
WHERE S3.sid IN (( SELECT R.sid
FROM Boats B, Reserves R
WHERE R.bid = B.bid AND B.color = 'red' )
INTERSECT
(SELECT R2.sid
FROM Boats B2, Reserves R2
WHERE R2.bid = B2.bid AND B2.color = 'green' ))
```

Our next example illustrates how the *division* operation in relational algebra can be expressed in SQL.

(Q2) Find the names of sailors who have reserved all boats.

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (( SELECT B.bid
FROM Boats B )
EXCEPT
(SELECT R.bid
FROM Reserves R
WHERE R.sid = S.sid ))
```

Notice that this query is correlated—for each sailor *S*, we check to see that the set of boats reserved by *S* includes all boats. An alternative way to do this query without using EXCEPT follows:

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS ( SELECT B.bid
FROM Boats B
WHERE NOT EXISTS ( SELECT R.bid
```

```
FROM Reserves R
WHERE R.bid = B.bid
AND R.sid = S.sid ))
```

Intuitively, for each sailor we check that there is no boat that has not been reserved by this sailor.

AGGREGATE OPERATORS

We now consider a powerful class of constructs for computing *aggregate values* such as MIN and SUM. These features represent a significant extension of relational algebra. SQL supports five aggregate operations, which can be applied on any column, say A, of a relation:

1. COUNT ([DISTINCT] A): The number of (unique) values in the A column.
2. SUM ([DISTINCT] A): The sum of all (unique) values in the A column.
3. AVG ([DISTINCT] A): The average of all (unique) values in the A column.
4. MAX (A): The maximum value in the A column.
5. MIN (A): The minimum value in the A column.

Note that it does not make sense to specify DISTINCT in conjunction with MIN or MAX (although SQL-92 does not preclude this).

(Q) Find the average age of all sailors.

```
SELECT AVG (S.age)
FROM Sailors S
```

On instance *S*3, the average age is 37.4. Of course, the WHERE clause can be used to restrict the sailors who are considered in computing the average age:

(Q) Find the average age of sailors with a rating of 10.

```
SELECT AVG (S.age)
FROM Sailors S
WHERE S.rating = 10
```

There are two such sailors, and their average age is 25.5. MIN (or MAX) can be used instead of AVG in the above queries to find the age of the youngest (oldest) sailor.

(Q) Find the name and age of the oldest sailor. Consider the following attempt to answer this query:

```
SELECT S.sname, MAX (S.age)
FROM Sailors S
```

The intent is for this query to return not only the maximum age but also the name of the sailors having that age. However, this query is illegal in SQL—if the SELECT clause uses an aggregate operation, then it must use *only* aggregate operations unless the query contains a GROUP BY clause! Thus, we cannot use MAX (S.age) as well as S.sname in the SELECT clause. We have to use a nested query to compute the desired answer to above question:

```
SELECT S.sname, S.age
FROM Sailors S
WHERE S.age = ( SELECT MAX (S2.age)
FROM Sailors S2 )
```

Observe that we have used the result of an aggregate operation in the subquery as an argument to a comparison operation. Strictly speaking, we are comparing an age value with the result of the subquery, which is a relation. However, because of the use of the aggregate operation, the subquery is guaranteed to return a single tuple with a single field, and SQL converts such a relation to a field value for the sake of the comparison. The following equivalent query for above question is legal in the SQL-92 standard but is not supported in many systems:

```
SELECT S.sname, S.age
FROM Sailors S
WHERE ( SELECT MAX (S2.age)
FROM Sailors S2 ) = S.age
```

We can count the number of sailors using COUNT. This example illustrates the use of * as an argument to COUNT, which is useful when we want to count all rows.

(Q) Count the number of sailors.

```
SELECT COUNT (*)
FROM Sailors S
```

We can think of * as shorthand for all the columns (in the cross-product of the fromlist in the FROM clause). Contrast this query with the following query, which computes the number of distinct sailor names. (Remember that *sname* is not a key!)

(Q) Find the names of sailors who are older than the oldest sailor with a rating of 10.

```
SELECT S.sname
FROM Sailors S
WHERE S.age > ( SELECT MAX ( S2.age )
FROM Sailors S2
WHERE S2.rating = 10 )
```

On instance *S*3, the oldest sailor with rating 10 is sailor 58, names of older sailors are Bob, Dustin, Horatio, and Lubber. could alternatively be written as follows:

```
SELECT S.sname FROM Sailors S
WHERE S.age > ALL ( SELECT S2.age
FROM Sailors S2
WHERE S2.rating = 10 )
```

The GROUP BY and HAVING Clauses

We want to apply aggregate operations to each of a number of groups of rows in a relation, where the number of groups depends on the relation instance (i.e., is not known in advance). For example, consider the following query.

(Q1) Find the age of the youngest sailor for each rating level.

If we know that ratings are integers in the range 1 to 10, we could write 10 queries of the form:

```
SELECT MIN (S.age)
FROM Sailors S
WHERE S.rating = i
```

where *i* = 1, 2, . . . , 10. Writing 10 such queries is tedious. More importantly, we may not know what rating levels exist in advance. To write such queries, we need a major extension to the basic SQL query form, namely, the GROUP BY clause. In fact, the extension also includes an optional HAVING clause that can be used to specify qualifications over groups (for example, we may only be interested in rating levels > 6). The general form of an SQL query with these extensions is:

```
SELECT [ DISTINCT ] select-list
FROM from-list
WHERE qualification
GROUP BY grouping-list
HAVING group-qualification
```

Using the GROUP BY clause, we can write Q31 as follows:

```
SELECT S.rating, MIN (S.age)
FROM Sailors S
GROUP BY S.rating
```

Let us consider some important points concerning the new clauses:
The select-list in the SELECT clause consists of
(1) a list of column names and
(2) a list of terms having the form aggop (*column-name*) AS *new-name*. The optional AS *new-name* term gives this column a name in the table that is the result of the query.
Any of the aggregation operators can be used for aggop.Every column that appears in (1)must also appear in grouping-list. The reason is that each row in the result of the query corresponds to one *group*, which is a collection of rows that agree on the values of columns in grouping-list. If a column appears in list (1), but not in grouping-list, it is not clear what value should be assigned to it in an answer row. The expressions appearing in the group-qualification in the HAVING clause must have a *single* value per group. The intuition is that the HAVING clause determines whether an answer row is to be generated for a given group. Therefore, a column appearing in the group-qualification must appear as the argument to an aggregation operator, or it must also appear in grouping-list. If the GROUP BY clause is omitted, the entire table is regarded as a single group. We will explain the semantics of such a query through an example. Consider the query:

Q) Find the age of the youngest sailor who is eligible to vote (i.e., is at least 18 years old) for each rating level with at least two such sailors.

```
SELECT S.rating, MIN (S.age) AS minage
FROM Sailors S
WHERE S.age >= 18
GROUP BY S.rating
HAVING COUNT (*) > 1
```

Extending the conceptual evaluation strategy presented previously, we proceed as follows. The first step is to construct the cross-product of tables in the from-list. Because the only relation in the from-list in Query above is Sailors, the result is just the instance shown in Figure below.

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horati	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure : Instance *S*3 of Sailors

The second step is to apply the qualification in the WHERE clause, *S.age >= 18*. This step eliminates the row ⟨71*, zorba,* 10*,* 16⟩. The third step is to eliminate unwanted columns. Only columns mentioned in the SELECT clause, the GROUP BY clause, or the HAVING clause are necessary, which means we can eliminate *sid* and *sname* in our example. The result is shown in Figure below. The fourth step is to sort the table

Rating	minage
3	25.5
7	35.0
8	25.5

Figure: Final Result in Sample Evaluation

Order BY

- Whenever we want to sort the records based on the columns stored in the tables of the SQL database, then we consider using the ORDER BY clause in SQL.
- The ORDER BY clause in SQL will help us to sort the records based on the specific column of a table. This means that all the values stored in the column on which we are applying ORDER BY clause will be sorted, and the corresponding column values will be displayed in the sequence in which we have obtained the values in the earlier step.
- Using the ORDER BY clause, we can sort the records in ascending or descending order as per our requirement. The records will be sorted in ascending order whenever the ASC keyword is used with ORDER by clause. DESC keyword will sort the records in descending order.
- *If no keyword is specified after the column based on which we have to sort the records, in that case, the sorting will be done by default in the ascending order.*

Syntax to sort the records in ascending order:

1. **SELECT** ColumnName1,...,ColumnNameN **FROM** TableName **ORDER BY** ColumnName **ASC**;

Syntax to sort the records in descending order:

1. **SELECT** ColumnName1,...,ColumnNameN **FROM** TableName **ORDER BY** ColumnNameDESC;

Syntax to sort the records in ascending order without using ASC keyword:

1. **SELECT** ColumnName1,...,ColumnNameN **FROM** TableName **ORDER BY** ColumnName;

NULL VALUES

We have assumed that column values in a row are always known. In practice column values can be unknown. For example, when a sailor, say Dan, joins a yacht club, he may not yet have a rating assigned. Since the definition for the Sailors table has a *rating* column, what row should we insert for Dan? What is needed here is a special value that denotes *unknown*. Suppose the Sailor table definition was modified to also include a *maiden-name* column. However, only married women who take their husband's last name have a maiden name. For single women and for men, the *maidenname* column is *inapplicable*. Again, what value do we include in this column for the row representing Dan? SQL provides a special column value called *null* to use in such situations. We use *null* when the column value is either *unknown* or *inapplicable*. Using our Sailor table definition, we might enter the row ⟨98*, Dan, null,* 39⟩ to represent Dan. The presence of *null* values complicates many issues, and we consider the impact of *null* values on SQL in this section.

Comparisons Using Null Values

Consider a comparison such as *rating = 8*. If this is applied to the row for Dan, is this condition true or false? Since Dan's rating is unknown, it is reasonable to say that this comparison should evaluate to the value unknown. In fact, this is the case for the comparisons *rating* > 8 and *rating* < 8 as well. Perhaps less obviously, if we compare two *null* values using *<, >,* =, and so on, the result is always unknown. For example, if we have *null* in two distinct rows of the sailor relation, any comparison returns unknown.

SQL also provides a special comparison operator IS NULL to test whether a column value is *null*; for example, we can say *rating* IS NULL, which would evaluate to true on the row representing Dan. We can also say *rating* IS NOT NULL, which would evaluate to false on the row for Dan.

Logical Connectives AND, OR, and NOT

Now, what about boolean expressions such as *rating* = 8 OR *age* < 40 and *rating* = 8 AND *age* < 40? Considering the row for Dan again, because *age* < 40, the first expression evaluates to true regardless of the value of *rating*, but what about the second? We can only say unknown. But this example raises an important point—once we have *null* values, we must define the logical operators AND, OR, and NOT using a *three-valued* logic in which expressions evaluate to true, false, or unknown. We extend the usual interpretations of AND, OR, and NOT to cover the case when one of the arguments is unknown as follows. The expression NOT unknown is defined to be unknown. OR of two arguments evaluates to true if either argument evaluates to true, and to unknown if one argument evaluates to false and the other evaluates to unknown. (If both arguments are false, of course, it evaluates to false.) AND of two arguments evaluates to false if either argument evaluates to false, and to unknown if one argument evaluates to unknown and the other evaluates to true or unknown. (If both arguments are true, it evaluates to true.)

INTRODUCTION TO VIEWS

A view is a table whose rows are not explicitly stored in the database but are computed as needed

from a view definition. Consider the Students and Enrolled relations. Suppose that we are often interested in finding the names and student identifiers of students who got a grade of B in some course, together with the cid for the course. We can de ne a view for this purpose. Using SQL-92 notation:

```
CREATE VIEW B-Students (name, sid, course)
AS SELECT S.sname, S.sid, E.cid
FROM Students S, Enrolled E
WHERE S.sid = E.sid AND E.grade = `B'
```

The view B-Students has three elds called *name*, *sid*, and *course* with the same domains as the fields *sname* and *sid* in *Students* and *cid* in Enrolled. (If the optional arguments *name*, *sid*, and *course* are omitted from the CREATE VIEW statement, the column names *sname*, *sid*, and *cid* are inherited.) This view can be used just like a base table, or explicitly stored table, in defining new queries or views. Given the instances of Enrolled and Students shown in Figure 3.4, B-Students contains the tuples shown in Figure below. Conceptually, whenever B-Students is used in a query, the view definition is first evaluated to obtain the corresponding instance of B-Students, and then the rest of the query is evaluated treating B-Students like any other relation referred to in the query.

Name	sid	course
Jones	53666	History105
Guldu	53832	Reggae203

Updates on Views

The motivation behind the view mechanism is to tailor how users see the data. Users should not have to worry about the view versus base table distinction. This goal is indeed achieved in the case of queries on views; a view can be used just like any other relation in defining a query. However, it is natural to want to specify updates on views as well. Here, unfortunately, the distinction between a view and a base table must be kept in mind. The SQL-92 standard allows updates to be specified only on views that are de ned on a single base table using just selection and projection, with no use of aggregate operations. Such views are called updatable views. This definition is oversimplified, but it captures the spirit of the restrictions. An update on such a restricted view can always be implemented by updating the underlying base table in an unambiguous way.

Consider the following view:

```
CREATE VIEW GoodStudents (sid, gpa)
AS SELECT S.sid, S.gpa
FROM Students S
WHERE S.gpa > 3.0
```

An important observation is that an INSERT or UPDATE may change the underlying base table so that the resulting (i.e., inserted or modifed) row is not in the view! For example, if we try to insert a row *h51234, 2.8i* into the view, this row can be (padded with *null* values in the other fields of Students and then) added to the underlying Students table, but it will not appear in the GoodStudents view because it does not satisfy the view condition *gpa* > 3:0. The SQL-92 default action is to allow this insertion, but we can disallow it by adding the clause WITH CHECK OPTION to the definition of the view. We caution the reader that when a view is de ned in terms of another view, the inter-action between these view

definitions with respect to updates and the CHECK OPTION clause can be complex; we will not go into the details.

DESTROYING/ALTERING TABLES AND VIEWS

If we decide that we no longer need a base table and want to destroy it (i.e., delete all the rows *and* remove the table definition information), we can use the DROP TABLE command. For example, DROP TABLE Students RESTRICT destroys the Students table unless some view or integrity constraint refers to Students; if so, the command fails. If the keyword RESTRICT is replaced by CASCADE, Students is dropped and any referencing views or integrity constraints are (recursively) dropped as well; one of these two keywords must always be specified. A view can be dropped using the DROP VIEW command, which is just like DROP TABLE. ALTER TABLE modifies the structure of an existing table. To add a column called *maiden-name* to Students, for example, we would use the following command:

ALTER TABLE Students
ADD COLUMN maiden-name CHAR(10)

The definition of Students is modified to add this column, and all existing rows are padded with *null* values in this column. ALTER TABLE can also be used to delete columns and to add or drop integrity constraints on a table; we will not discuss these aspects of the command beyond remarking that dropping columns is treated very similarly to dropping tables or view.

TRIGGERS

A trigger is a set of SQL statements that reside in system memory with unique names. It is a specialized category of stored procedure that is called automatically when a database server event occurs. Each trigger is always associated with a table.

A trigger description contains three parts:

- **Event**: A change to the database that activates the trigger.
- **Condition:** A query or test that is run when the trigger is activated.
- **Action:** A procedure that is executed when the trigger is activated and its condition is true.

When we use triggers?

Triggers will be helpful when we need to execute some events automatically on certain desirable scenarios. **For example**, we have a constantly changing table and need to know the occurrences of changes and when these changes happen. If the primary table made any changes in such scenarios, we could create a trigger to insert the desired data into a separate table.

Syntax of Trigger

We can create a trigger in SQL Server by using the **CREATE TRIGGER** statement as follows:

create trigger [trigger_name]

[before | after]

{insert | update | delete}

on [table_name]

[for each row]

[trigger_body]

Explanation of Syntax

Create trigger [trigger_name]: Creates or replaces an existing trigger with the trigger_name.

[before | after]: This specifies when the trigger will be executed.

{insert | update | delete}: This specifies the DML operation.

On [table_name]: This specifies the name of the table associated with the trigger.

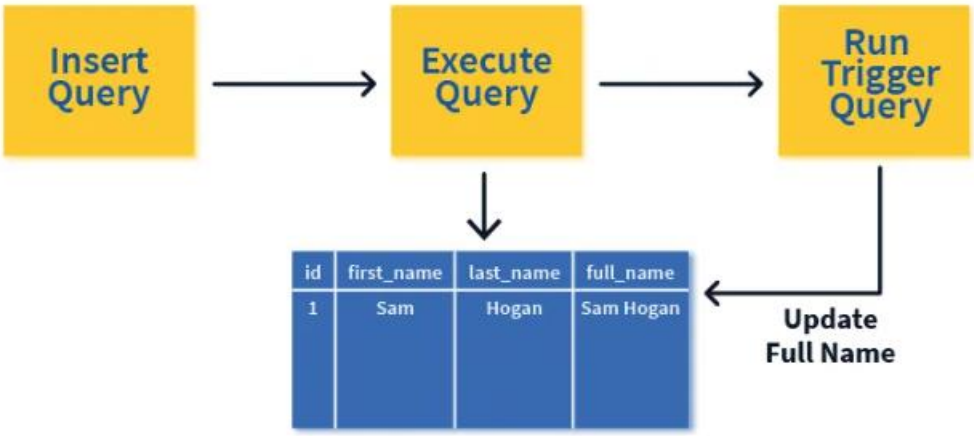
[for each row]: This specifies a row-level trigger, i.e., the trigger will be executed for each affected row.

[trigger_body]: This provides the operation to be performed as the trigger is fired

Example:

CREATE TABLE student(Id integer PRIMARY KEY, first_name varchar(50), last_name varchar(50), full_name varchar(50));

Here we will create a trigger to fill in the full name by concatenating the first and last names. So while inserting the values, we will only feed the first name and last name, and we will expect the trigger to automatically update each row with an additional column attribute bearing the full name.



Creation of Trigger:

create trigger student_name

after INSERT

on

student

for each row

```
BEGIN
  UPDATE student set full_name = first_name || ' ' || last_name;
END;
```

Here we can understand from the trigger query we have set a trigger after an insert is made to the table student. Once the insert is done, this procedure will be fired, which will run an update command to update the students' full names.

Here we can see since we had an update statement in the trigger procedure in query 2, the full names are automatically updated immediately after the inserts are done

Here we can understand from the trigger query we have set a trigger after an insert is made to the table student. Once the insert is done, this procedure will be fired, which will run an update command to update the students' full names.

Drop Triggers in SQL

The deletion of a trigger is quite straightforward. You need to run a simple query to delete a trigger from the database.

```
DROP TRIGGER student_name;
```

UNIT-3

RELATIONAL DATABASE DESIGN

Redundancy and Problems Caused by Redundancy

Redundancy in DBMS is the problem that arises when the database is not normalized. It is the concept of storing multiple copies of the same data in different parts of the database.

Example for Redundancy in DBMS?

Let's understand the concept of redundancy in DBMS with a simple student table.

student_id	student_name	student_age	dept_id	dept_name	dept_head
1	Tony Stark	18	100	Computer Science	Steve Rogers
2	Thor Odinson	18	100	Computer Science	Steve Rogers

In this student table, we have repeated the same department details, dept_id, dept_name, and dept_head in every student record. This causes redundancy in the student table.

How does Data Redundancy Occur?

Data redundancy in DBMS may occur due to any of the below reasons.

1. The database is not normalized through DBMS normalization.
2. The same data is stored in multiple places by multiple systems causing redundancy in data.
3. Mistake during database design causes the same data to be stored multiple times.

Problems caused by redundancy in Database

Redundancy in DMBS can cause several problems while performing operations on data such as insert, delete, and update. Let's use the below student table to understand **insertion**, **updation**, and **deletion** anomalies.

student_id	student_name	student_age	dept_id	dept_name	dept_head
1	Tony Stark	18	100	Computer Science	Steve Rogers
2	Thor Odinson	18	100	Computer Science	Steve Rogers
3	Bruce Banner	18	101	Mechanical	Natasha Romanoff

Insertion Anomaly

An insertion anomaly occurs when specific details cannot be inserted into the database without the other details.

Example: Without knowing the department details, we cannot insert the student details in the above table. The student details (student_id, student_name, and student_age) depends on the department details (dept_id, dept_name, and dept_head).

Deletion Anomaly

Deletion anomaly occurs when deleting specific details loses some unrelated information from the database.

Example: If we delete the student with student_id **3** from the above student table, we also lose the department details with dept_id **101**. Deleting student details result in losing unrelated department details.

Updation Anomaly

Updation anomaly occurs when there is data inconsistency resulting from a partial data update.

Example: We wanted to update the dept_head to **Peter Parker** for dept_id **101**; we need to update it in all places. If the update didn't occur in all the places (partial update), it may result in data inconsistency.

A DBMS's redundancy can lead to many issues, some of which are listed below:

1. **Inconsistent data:** Redundancy raises the possibility of data inconsistency. When the same information is stored in many locations, any update, deletion, or alteration made to one instance might not reliably propagate to all the redundant copies. Information that is erroneous and unreliable might result from these disparities and conflicts within the data.
2. **Issues with Data Integrity:** Maintaining data integrity is challenging in the presence of redundancy. Integrity violations may be caused by inconsistent updates or the failure to synchronize changes across redundant data instances. It becomes difficult to make sure that every redundant copy of the data is updated correctly and at the same time, raises the possibility of integrity problems in the DBMS.
3. **Increased Storage Requirements:** Redundant data requires more storage because it takes up more space. Multiple copies of the same data stored in the database might considerably increase its overall storage needs. In particular when working with big volumes of data, this results in higher storage costs, increased disc consumption, and significant performance problems.
4. **Update Anomalies:** Update anomalies can be brought on by redundancy and include insertion, deletion, and modification anomalies. For instance, if redundant data is not regularly updated, multiple copies of the same data may have different values. Due to this, it becomes difficult to manage the database's consistency and integrity, which increases the likelihood of data anomalies and errors.
5. **Reduced Performance:** A DBMS's performance may suffer as a result of redundancy. Due to the additional data that must be processed, increased storage requirements from redundant data lead to slower data retrieval times. Additional computational complexity is needed to preserve the consistency and integrity of redundant data, which slows down query execution times and lowers system performance as a whole.

6. **Increased Complexity:** Redundancy complicates database design and management procedures, which increases complexity. It takes more time and effort to manage redundant data. When redundant data is present, it becomes more difficult to implement database schema modifications, enforce data constraints, and guarantee data accuracy. As redundancy grows, the system's upkeep and administration become more difficult.
7. **Increased Maintenance Efforts:** Redundant data requires more work to maintain, which increases maintenance costs. The likelihood of mistakes and inconsistencies increases while updating, backing up, and migrating data across numerous redundant data instances. It takes more time and resources to complete maintenance chores as they get more complicated.

How To Avoid Redundancy in DBMS?

Redundancy in DBMS can be avoided by following the below approaches.

- Redundancy in DBMS can be avoided by normalizing the data through database normalization.
- Redundancy can be avoided using Master Data. Master data is a single source of data accessed by several applications and systems.
- Proper database architecture design can avoid data redundancy.
- Decomposition in SQL involves breaking a single relation into multiple smaller relations to eliminate Redundancy

Functional Dependency

The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table.

$X \rightarrow Y$

The left side of FD is known as a determinant, the right side of the production is known as a dependent.

For example:

Assume we have an employee table with attributes: Emp_Id, Emp_Name, Emp_Address.

ere Emp_Id attribute can uniquely identify the Emp_Name attribute of employee table because if we know the Emp_Id, we can tell that employee name associated with it.

Functional dependency can be written as:

Emp_Id → Emp_Name

We can say that Emp_Name is functionally dependent on Emp_Id.

Types of Functional dependency

1. Trivial functional dependency

- A → B has trivial functional dependency if B is a subset of A.
- The following dependencies are also trivial like: A → A, B → B

2. Non-trivial functional dependency

- A → B has a non-trivial functional dependency if B is not a subset of A.
- When A intersection B is NULL, then A → B is called as complete non-trivial.

Completely Non-Trivial Functional Dependency

A completely non-trivial functional dependency occurs when A intersection B is null in –

A -> B

Functional Dependencies and its reasoning in DBMS

Closure of a Set of FDs

The set of all FDs implied by a given set *F* of FDs is called the **closure of F** and is denoted as *F*+. An important question is how we can **infer**, or compute, the closure of a given set *F* of FDs. The answer is simple and elegant. The following three rules, called **Armstrong's Axioms**, can be applied repeatedly to infer all FDs implied by a set *F* of FDs. We use *X*, *Y*, and *Z* to denote *sets* of attributes over a relation schema *R*:

- **Reflexivity:** If $X \supseteq Y$, then $X \rightarrow Y$.
- **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z.
- **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Armstrong's Axioms are **sound** in that they generate only FDs in *F*+ when applied to a set *F* of FDs. They are **complete** in that repeated application of these rules will generate all FDs in the closure *F*+. (We will not prove these claims.) It is convenient to use some additional rules while reasoning about *F*+:

- **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.

HOW TO FIND ATTRIBUTE CLOSURE USING FUNCTIONAL DEPENDENCY

For a Given set of Functional dependencies, the attribute closure of an attribute A will be a set S determined as below:

1. Add A to the set S.
2. Add all the attributes which are derived from A to the set S
3. Add all the attributes which are functionally determined by the attributes of set S, recursively.

Attribute closure is denoted by A^+

Example:

Given a Relation R(A, B, C, D, E) {AB -> CD, D -> E, A -> C,B -> D}.

we can determine A^+ as:

1. Add A to the set S = {A}.
2. get all the attribute which are derived from A
    - A->C (From given Functional Dependencies)
    - Add C to the set. Now S will contain {A,C}
3. No other attribute can be derived from A and C hence

A^+= {A,C}.

we can determine B^+ as:

1. Add B to the set S = {B}.
2. get all the attribute which are derived from B
    - B->D (From given Functional Dependencies)
    - Add D to the set. Now S will contain {B,D}
3. get all the attribute which are either derived by D or the combination of BD
    - D->E
    - Add E to the Set. Now S will contain {B,D,E}
4. No other attribute can be derived from B,D,E or with their combination hence

B^+= {B,D,E}.

Similarly we can get closure of any attribute.